

A type-preserving compiler from a systems language to DTAL

Athena Eng

10 October 2025

1 Introduction

Systems programming languages such as C and C++ offer the programmer granular control over hardware and efficient use of resources, making them appropriate for performance-critical applications. However, these languages are fundamentally memory-unsafe and can lead to security vulnerabilities that go unnoticed in production code. For example, the Heartbleed bug [1] was the result of an out-of-bounds read, which led to sensitive data being leaked.

There are several mechanisms that modern languages implement to address problems such as out-of-bounds access. Languages such as Python and Java use runtime checks and garbage collection to ensure memory safety guarantees, but these methods incur overhead that can degrade performance. The Rust programming language enforces compile-time safety using language features such as ownership and borrow-checking to prevent undefined behaviour (e.g. use-after-free and double free) that is prevalent in C and C++.

While Rust does provide a strong solution for preserving safety guarantees while maintaining performance comparable to C and C++, its correctness relies on the assumption that the compiler implementation is correct and preserves the semantics of the high-level (source) language. However, there may be situations in which a compiler for some given source language is not trusted. Projects such as CompCert [2] and CakeML [3] have developed verified compilers that ensure compiled programs behave exactly as the source program was defined.

This dissertation project aims to move the locus of trust from the compiler to a simple, independent verifier. Formal compiler verification is complex and requires a lot of work whereas a verifier can be a small and simple trusted base. Proving the full semantic equivalence between source and target programs will be beyond the scope of this project. This project will implement a toolchain that uses type-preserving compilation to guarantee safety properties and statically verify the absence of certain unsafe behaviours and classes of runtime errors. A type-preserving compiler translates well-typed source code into verifiable target code, which for this project will be a typed assembly language. The typed assembly language will use features inspired by DTAL [4], where a register's type carries information about its value (e.g. the length of an array). Future work on this project could involve developing a formally verified compiler for the target assembly lan-

guage such that both correctness and safety are achieved.

1.1 Dependent Type Theory

Dependent type theory (DTT) is a flavour of type theory that introduces dependent types, a type whose definition depends on a value. Dependent types also feature in Martin-Löf type theory, which can serve as an alternative foundation to mathematics. The theory promotes a view of a type as a mathematical proposition, where a value of that type would be a proof of that proposition. This correspondence between programs and proofs is known as the Curry-Howard isomorphism.

Type-checking DTT with extensional equality¹ is actually undecidable. The work of Xi & Pfenning, which was inspiration for this project, implements a dependently type assembly language (DTAL) using a restricted form of DTT based on the decidable logic of linear integer arithmetic. Types are not first-class citizens as one might observe in proof languages such as Agda, but rather exist at compile-time to verify program properties. This model allows the type system to be powerful enough to express properties like loop invariants and verify them statically. Using dependent types specifically enables certain compiler optimisations that would not be possible using a more traditional type system [5]. For example, runtime bounds checks can be transformed into compile-time proofs without the overhead.

1.2 Safety Properties

The dependent type system of this project is intended to provide a framework for proving program properties, with a focus on proving structural properties as described in the following.

Spatial Memory Safety

- Guarantee that every access to a memory buffer is within its legal bounds.
- We define an array's type as a dependent pair (int, m) , where m is the array's size. Any indexing operation generates a proof obligation (e.g. $0 \leq i < m$), which must be solved by the compiler frontend.

Control-Flow Integrity

- Guarantee that branch instructions only target valid destinations.
- The DTAL labels destinations with a type to represent machine state, which will be compared with the current state at a branch instruction that jumps there, which involves subtype checking.

¹extensional equality means that definitional equality is implied by propositional equality.

Integer Overflows

- Guarantees that integer arithmetic operations will not overflow or underflow.
- Range/refinement types can be used to guarantee integers stay within a range. These types can also be used for refining types in select blocks e.g. `{i:int | i < 10}`).

Preconditions and Postconditions

- Guarantees that a function is only ever called with inputs that satisfy given constraints.
- The HLL will have a language feature to specify preconditions (e.g. `require(m <= n)` on a function defines a Π -type). The compiler frontend then must prove this predicate holds true at every call.

The dependent type system could also be extended to prove more complex concurrent and behavioural properties. The following properties move beyond the core project.

Data Race Freedom

- Guarantees that two threads cannot concurrently access the same memory location if at least one of those accesses is a write.
- We could use ownership and linear/affine types similar to Rust's concurrency model and `send` and `sync` types for passing data between threads.

Information Flow Control

- Guarantees that sensitive information such as private keys and passwords cannot leak into public channels (e.g. log files, network sockets).
- We could extend the type system by incorporating security labels into the type system and define a security lattice that defines legal flow of information.

Communication Protocols

- Guarantees that two threads conform to a given message-passing protocol.
- Session types can be used for defining sequences of communication over a channel by describing the types that are expected to be sent/received during communication. Types evolve to reflect the next state of the protocol.

2 Structure and Substance of the Project

The core of this project will involve specifying a high-level systems language as the source language, low-level target language similar to DTAL as well as intermediate languages; implementation of a compiler for compiling the systems language to DTAL; and a verifier to assert a number of properties and safety guarantees in the generated target code.

Additional features that could build on the foundation of the core project include an interpreter for the generated DTAL, extending the specified core languages and support for concurrency properties and behavioural correctness. The full list of proposed extensions for the project are listed in 4.2.

The implementation will be written in Rust, which offers safety, stability and high performance.

2.1 Main Components

Figure 1 shows the components of the project and their inputs and outputs. The core components include specification of the source and target languages, the compiler and the verifier, whereas the assembler and linker for generating an object file from the DTAL will be an extension.

As stated in the introduction, the goal of this project is not to produce a formally verified compiler. Hence, in the design of this system, the compiler is untrusted, though compiler implemented in this project should perform the essential function of faithfully encoding the high-level safety specifications of the source language as explicit type annotations in the low-level code. The trusted component is the verifier, which will take the DTAL file as input and independently check that claims expressed in the type-annotated instructions logically follow from the code’s execution. Hence, its correctness is paramount for providing a definite guarantee that programs are free from given classes of errors.

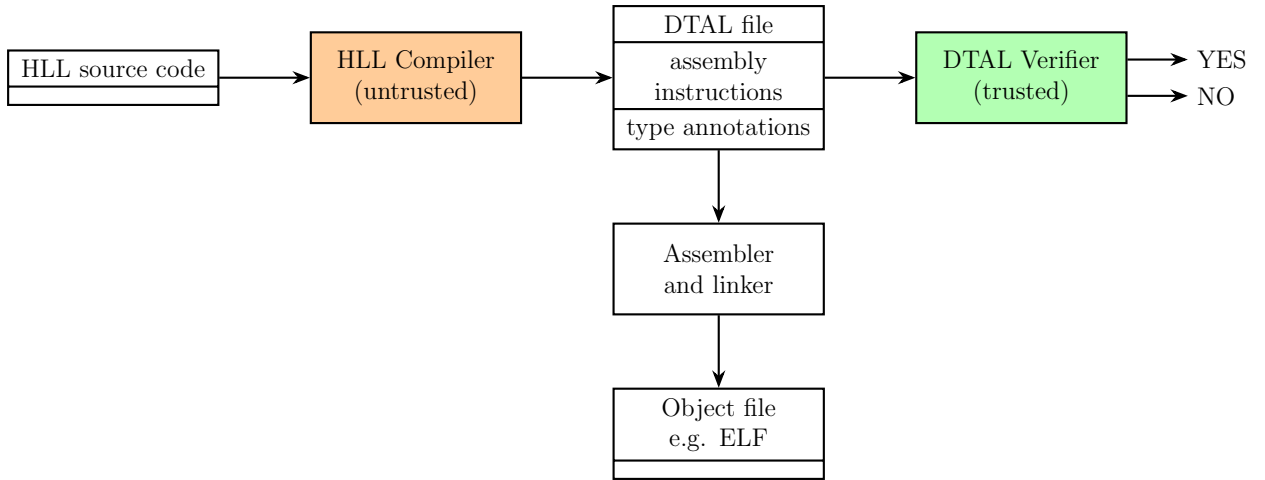


Figure 1: System model of the project

High-level Language Design

The source language will be a minimal systems language (consisting of C-like and Rust-like features) focused on demonstrating the core safety properties.

- Constructs:
 - first-order functions (**fn**)

- assignment (`let`)
- if/else
- for (`for i in 0..n`)
- preconditions (`requires(...)`)
- Types:
 - primitive types (`int`, `bool`)
 - fixed-size array (`[T; N]`, where `T` is the element type and `N` is the size)
 - immutable reference (`&T`)
 - mutable variable (`let mut x = ...`)
 - singleton types (`int(c)`)
 - range/refinement types (`{i: int | P(i)}`, where `P` is some proposition)

DTAL Design

The target DTAL will be a RISC-like instruction set but every instruction will have a formal typing rule so it can be verified.

ISA:

- Moving data (`mov`, `load`, `store`)
- Arithmetics (`add`, `sub`, `mul`)
- Control flow (`jmp`, `cmp`, `blt`, `bgt`, `beq`, `ret`)

Type system:

- primitive types
- dependent integer types
- pointer types (`ptr(T)`)
- state types for labels (`r1:T1, ...`)
- function types for entry points (`(args...) -> ret`)

Compiler Frontend

The first stage of the compiler frontend involves lexing and parsing, which will be done with the help of a parser generator/combinator to build an abstract syntax tree (AST). There are several existing Rust crates² that do this. The type-checker will be the main substance of work for the frontend. It will traverse the AST and apply the language's formal typing rules as well as generating and solving proof obligations. For example:

²A Rust crate is a compilation unit that the Rust compiler processes at a time e.g. binaries, libraries

- For an array access `arr[i]`, the types of `arr` (e.g. `&(int, m)`) and `i` are synthesised and the proof obligation `i >= 0 && i < m` is generated.
- For a function with a `requires(n <= m)` clause, the function arguments are substituted into the clause to generate a proof obligation.

After the AST has been fully traversed, the frontend will have a set of logical constraints that must hold true. The constraint solving will be done with the help of an external SMT solver such as Z3. Finally, the AST will be annotated with the verified types.

Compiler Backend

The compiler backend is responsible for translating the AST from the frontend into a linear sequence of well-typed DTAL. Note that the compiler I develop will not be formally verified but it should perform proof transformations faithfully, encoding the safety guarantees verified by the frontend as correct and explicit type annotations in the target DTAL. The project's model assumes that the compiler could be incorrect, but the compiler delivered here aims to be as correct as possible by following standard engineering principles. Implementation will be semantics-driven with rigorous testing to maximise confidence of correctness.

The backend's workflow will be implemented as a multi-stage pipeline that will consist of:

- Lowering to a defined typed intermediate representation (IR) that will resemble a linear, three-address code representation structured as a CFG using simplifying assumptions e.g. (an infinite set of virtual registers).
- Instruction selection and typed code generation where each instruction in the IR is mapped to one or more DTAL instructions. This process should preserve and encode type information.
- Typed register allocation where a register allocation algorithm will be implemented that takes the type system into account.
- Emission of DTAL file containing the finalised instructions with annotations and labels.

Some examples of transformations to DTAL include:

- Arithmetic expression translation: we can derive resulting dependent types (e.g. for `add r3, r1, r2` where `r1:int(1)` and `r2:int(2)` the result register `r3` has type `int(3)`).
- Selection statement translation: we can generate explicit labels and conditional branch instructions. Refinement types can be used to refine the type of values in registers based on conditions (e.g. `r1:int` becomes `r1:{i:int | i < 10}` for a branch where the value in `r1 < 10`).
- For loop translation: loop invariants can be expressed as a type (e.g. `r1:{i:int | 0 <= i < n}`) and we can generate loop start/end labels.

DTAL Verifier

The DTAL verifier will take the generated assembly and verify its proofs. This is the root of trust in the system so it should be small and simple (relative to the compiler). As mentioned, the compiler is large and complex whereas it is far more tractable to formally verify the verifier. This helps achieve a level of certified safety in contexts such as supply chain security or platform providers where there may be zero trust in external toolchains.

The verifier will lex and parse the assembly to produce an AST for the DTAL grammar. From this, we build a control flow graph with annotated type information where each node is a basic block and the edges between blocks are the branch instructions. Using the CFG, we can execute an algorithm to perform data flow analysis that keeps track of the types of register values and prove that this is consistent with the type annotations. Some components and mechanisms used for the compiler frontend can be reused here e.g. verification of logical constraints via an SMT solver, AST traversal etc.

Within a block in the CFG, the verifier can perform actions on each instruction to check and update. For example, an instruction such as `mov r4, 0` will update the type state to express that `r4` is of the singleton type `int(0)`. For an instruction `load r3, [r1, r2]` where `r1` is a pointer to an array with type `ptr(array(int, m))` and `r2` is an index `nat` to access the array, the algorithm would need to check `m` against proofs in the typing state to ensure that the value in `r2` is less than `m`.

Given that the verifier should be trusted, it will need to be rigorously tested. It is intended to be small enough such that a full manual audit would be possible. The gold standard would be formal verification via a proof assistant, which would be a major extension to the project.

3 Starting Point

- The Part IB Compiler Construction, Introduction to Computer Architecture and Semantics of Programming Languages courses covered concepts relevant to this project.
- I have foundational knowledge of dependent type theory, having used it as the topic of an introductory technical talk I delivered.
- I have limited practical experience with proof-carrying code and using proof assistants.
- I do not have extensive practical experience with compiler development.
- I began learning the Rust programming language over the 2025 summer vacation and worked in a Rust codebase during my internship.

4 Success Criterion

4.1 Core

- High-level language with core language features (see 2.1).
- DTAL with core language features (see 2.1). The extended version of Xi and Pfennig’s paper [4] contains the full rules of a DTAL, which can be used as reference.
- Compiler frontend to fully support lexing, parsing and type-checking of the core high-level language and its type system. There are existing type-checkers for refinement type systems that can be used from reference.
- Compiler backend to fully support compilation of the core high-level language and reject unsafe programs at compile time for the core safety properties (see 1.2).
- Verifier that accepts assembly code generated from safe programs.

4.2 Extensions

- An interpreter and object file generator to produce an executable from the target DTAL.
- Support for modularised compilation, verification and linking.
- Widening the language and type system with more features (e.g. structs, generics, error handling, ownership, linear types).
- Prove some concurrency and behavioural correctness properties such that the verifier accepts a wider range of correct programs.

5 Evaluation Plan and Metrics

I will create a test suite to test the final system against safe and unsafe programs which should compile and verify, or be rejected by the DTAL verifier. With the language features defined in my core, this can write programs for performing calculations such as Fibonacci, factorial etc. and performing operations on vectors such as sorting. The Rust QuickCheck crate³ can be used for generating programs with predefined properties to test the toolchain.

I will also measure end-to-end compilation time in order to evaluate to what extent ensuring correctness impacts performance. I will also perform a qualitative assessment of the range of safety properties that can be expressed or enforced by the language, and how this translates to its real-world application.

³<https://docs.rs/quickcheck/latest/quickcheck/>

6 Timetable and Milestones

6.1 Michaelmas Term

- **16 Oct. - 29 Oct.**

- Design and formally define core high-level language, DTAL and type system.
- **Milestone:** Send formal definition of semantics and typing rules of the HLL and DTAL to supervisor for review and beginnings of their in-code implementation.

- **30 Oct. - 12 Nov.**

- Finish with design and implementation of languages.
- Implement lexer and parser (using parser generator/combinator).
- **Milestone:** Send full design of languages with lexer and parser for the HLL to supervisor for review.

- **13 Nov. - 26 Nov.**

- Revision for DSP module written test.
- DSP module written test (21 Nov.)
- Start implementing frontend type-checker (if time allows).
- **Milestone:**

- **27 Nov. - 10 Dec.**

- Implement the frontend type-checker with proof obligation checking using a logical constraint solver.
- **Milestone:** Implemented type-checker that applies formal typing rules with passing unit tests covering the major blocks of logic; compiler frontend implementation complete and ready for supervisor review.

6.2 Michaelmas Vacation

- **11 Dec. - 24 Dec.**

- Break
- Design IR and implement translation of HLL to IR.
- **Milestones:** Complete design of IR and translator.

- **25 Dec. - 7 Jan.**

- Implement translation of IR to DTAL.
- **Milestones:** Complete translator that lowers IR to DTAL and preserves type information.

- **8 Jan. - 21 Jan.**

- Implement register allocation algorithm.
- Implement emitter to output final DTAL code.
- **Milestone:** Majority of compiler backend implementation complete.

6.3 Lent Term

- **22 Jan. - 4 Feb.**

- Progress report and presentation.
- Start implementing verifier.
- **Milestones:** Delivered progress report and presentation and have beginnings of a verifier implementation (lexer, parser, CFG construction).

- **5 Feb. - 18 Feb.**

- Plan structure of dissertation.
- Finish verifier implementation (data flow analysis logic)
- **Milestone:** Verifier implementation complete.

- **19 Feb. - 3 Mar.**

- Write dissertation introduction.
- Begin end-to-end testing and evaluation.
- **Milestone:** Submit draft of dissertation to supervisor and obtain some evaluation results.

- **4 Mar. - 17 Mar.**

- Write dissertation preparation section.
- Buffer time.
- **Milestone:** Submit draft of dissertation to supervisor.

6.4 Easter Vacation

- **18 Mar. - 31 Mar.**

- Write dissertation implementation section.
- Extensions
- **Milestone:** Submit draft of dissertation to supervisor.

- **1 Apr. - 14 Apr.**

- Write dissertation evaluation section.
- Extensions
- **Milestone:** Submit full draft of dissertation to supervisor.

- **15 Apr. - 28 Apr.**

- Work on drafts of dissertations.
- Extensions
- **Milestone:** Dissertation approved by supervisor.

6.5 Easter Term

- **29 Apr. - 10 May**

- Final checks.
- **Milestone:** Dissertation ready for submission.

7 Resources Declaration

I will be developing on my laptop for this project. It has a 13th Gen Intel(R) Core(TM) i7-1365U processor and 32 GB RAM. I will make regular backups of the code to a private GitHub repository and a USB drive.

References

- [1] The MITRE Corporation. CVE-2014-0160. Common Vulnerabilities and Exposures, April 2014. URL: <https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2014-0160>.
- [2] Xavier Leroy. Formal verification of a realistic compiler. *Commun. ACM*, 52(7):107–115, July 2009.
- [3] Ramana Kumar, Magnus O. Myreen, Michael Norrish, and Scott Owens. CakeML: a verified implementation of ML. In *Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL ’14, page 179–191, New York, NY, USA, 2014. Association for Computing Machinery.
- [4] Hongwei Xi and Robert Harper. A dependently typed assembly language. *SIGPLAN Not.*, 36(10):169–180, October 2001.
- [5] Greg Morrisett, David Walker, Karl Crary, and Neal Glew. From system F to typed assembly language. *ACM Trans. Program. Lang. Syst.*, 21(3):527–568, May 1999.